

Trabalho Prático Individual I: Estruturas em Árvores Avançadas

Modelagem, Implementação e Análise Comparativa de Estruturas em Árvore Especializadas

Gabriel Alves Faria

Graduando em Engenharia de Computação

Centro Federal de Educação Tecnológica de Minas Gerais (CEFET-MG – Campus V)

Divinópolis, Brasil

gabrielalvesfaria@gmail.com

Abstract—Este trabalho apresenta o estudo, implementação em C++17 e análise comparativa de cinco estruturas de dados em árvore especializadas: Trie, Árvore Patricia (Radix Tree compacta), Árvore Splay, Árvore Treap e KD-Tree. Tais estruturas abordam limitações das árvores binárias de busca (BST) e AVL em cenários práticos como indexação por prefixos, localidade de acesso, balanceamento probabilístico e particionamento espacial multidimensional. O trabalho integra fundamentação teórica, rastreamento visual das transformações estruturais, análise formal de complexidade assintótica e avaliação experimental de desempenho em diferentes padrões e tamanhos de dados (N até 100.000). Os resultados práticos confirmaram a convergência com as estimativas teóricas, evidenciando o impacto de constantes ocultas e decisões de projeto no desempenho observado.

Index Terms—Estruturas de Dados, Árvores Avançadas, Trie, Árvore Patricia, Árvore Splay, Árvore Treap, KD-Tree, Análise de Complexidade.

I. INTRODUÇÃO E FUNDAMENTAÇÃO TEÓRICA

As árvores binárias de busca (BST) e suas variantes balanceadas, como a árvore AVL, constituem estruturas fundamentais para a organização e recuperação eficiente de dados ordenados. Entretanto, tais estruturas apresentam limitações em contextos específicos, como a representação de conjuntos de strings com prefixos compartilhados, a adaptação dinâmica a padrões de acesso desiguais, o balanceamento probabilístico com baixo custo de implementação e a organização de dados multidimensionais. Nesse contexto, estruturas especializadas – Trie, Árvore Patricia, Árvore Splay, Árvore Treap e KD-Tree – foram concebidas para atender a demandas particulares, cada uma explorando estratégias distintas de organização, balanceamento e indexação. Este trabalho estuda, implementa e compara essas cinco estruturas, relacionando seus fundamentos teóricos ao comportamento observado experimentalmente.

A. Trie (Árvore de Prefixos)

A Trie, proposta originalmente por Fredkin [3], é uma estrutura hierárquica destinada à representação eficiente de conjuntos de strings. Diferentemente das árvores de busca convencionais, cada nó da Trie não armazena uma chave completa, mas sim um símbolo do alfabeto utilizado, de modo que o caminho percorrido da raiz até um nó marcado

como terminal corresponde a uma chave armazenada na estrutura. Essa organização permite que chaves com prefixos comuns compartilhem os mesmos nós iniciais, reduzindo a redundância de armazenamento em conjuntos de dados com alta sobreposição de prefixos.

A complexidade das operações fundamentais – inserção, busca e remoção – é da ordem de $O(m)$, em que m representa o comprimento da chave manipulada, sendo independente do número total de chaves armazenadas na estrutura. Essa característica torna a Trie particularmente adequada para aplicações como sistemas de autocompletar, corretores ortográficos, dicionários e processamento de linguagem natural. Como limitação, observa-se um consumo elevado de memória quando o alfabeto utilizado é extenso e os prefixos compartilhados entre as chaves são escassos, uma vez que cada símbolo distinto corresponde a um nó individual.

B. Árvore Patricia (Radix Tree Compacta)

A árvore Patricia, introduzida por Morrison [4], constitui uma variação compacta da Trie, concebida para eliminar a ineficiência decorrente de nós com um único filho. Nessa estrutura, sequências de nós unários são colapsadas em uma única aresta, rotulada por uma subcadeia de caracteres, de modo que cada nó interno passa a possuir, necessariamente, pelo menos dois filhos. Essa compactação assegura que o número de nós internos da estrutura seja proporcional ao número de chaves armazenadas, e não ao comprimento total dessas chaves, resultando em economia significativa de espaço em relação à Trie tradicional.

As operações de busca, inserção e remoção mantêm complexidade $O(m)$, sendo m o comprimento da chave buscada. A árvore Patricia é amplamente empregada em tabelas de roteamento IP, nas quais se realiza a busca pelo prefixo mais longo correspondente (*longest prefix match*), bem como em índices compactos para conjuntos esparsos de strings e em variantes como a Merkle Patricia Trie, empregada no armazenamento de estado em sistemas blockchain.

C. Árvore Splay

A árvore Splay, apresentada por Sleator e Tarjan [7], é uma árvore binária de busca autoajustável, caracterizada pela reorganização dinâmica de sua estrutura a cada operação de acesso. Após uma busca, inserção ou remoção, o nó acessado é deslocado até a raiz por meio de uma sequência de rotações – denominada *splaying* –, que contempla três casos fundamentais: *zig*, *zig-zig* e *zig-zag*, conforme a posição relativa do nó em relação a seu genitor e avô.

Diferentemente da árvore AVL, a Splay não mantém um fator de balanceamento explícito; o equilíbrio da estrutura resulta, de forma amortizada, das sucessivas reorganizações provocadas pelos acessos. Essa propriedade confere à árvore Splay uma característica de localidade temporal, na qual elementos acessados recentemente permanecem próximos da raiz, favorecendo sequências de acesso com padrões desiguais. A análise amortizada, fundamentada no método do potencial, demonstra que o custo por operação é $O(\log n)$, ainda que uma operação isolada possa, no pior caso, custar $O(n)$. Essa estrutura encontra aplicação em sistemas de cache e em outras estruturas de dados avançadas, como as *link-cut trees*.

D. Árvore Treap (Tree + Heap)

A árvore Treap combina as propriedades de uma árvore binária de busca com as de um heap, associando a cada nó, simultaneamente, uma chave e uma prioridade atribuída de forma aleatória. Enquanto as chaves respeitam a propriedade de ordenação característica das BSTs, as prioridades obedecem à propriedade de heap, o que garante, com alta probabilidade, uma estrutura balanceada. O fundamento conceitual da Treap remonta à noção de árvore cartesiana, descrita por Vuillemin [8], tendo sido formalizada e analisada probabilisticamente por Seidel e Aragon [6].

Para um conjunto fixo de chaves, a configuração da árvore é unicamente determinada pelas prioridades atribuídas, e rotações são empregadas para restaurar a propriedade de heap sempre que uma inserção ou remoção viola essa condição. As operações fundamentais apresentam complexidade esperada $O(\log n)$, decorrente diretamente da aleatoriedade das prioridades, o que confere à Treap um balanceamento probabilístico obtido com relativa simplicidade de implementação, em contraste com os mecanismos determinísticos da árvore AVL.

E. KD-Tree (*k*-dimensional Tree)

A KD-Tree, proposta por Bentley [1], é uma estrutura espacial destinada à organização de pontos em espaços multidimensionais. Cada nível da árvore realiza o particionamento do espaço segundo uma dimensão distinta – de forma cíclica entre as k dimensões –, dividindo o conjunto de pontos por meio de um hiperplano perpendicular ao eixo escolhido.

A construção da estrutura apresenta complexidade $O(n \log n)$, enquanto a busca pelo vizinho mais próximo (*nearest neighbor search*) alcança, em média, complexidade $O(\log n)$, podendo degradar-se para $O(n)$ em espaços de alta dimensionalidade, fenômeno conhecido como maldição da dimensionalidade. Operações de inserção e remoção, por sua

vez, tendem a desbalancear a estrutura ao longo do tempo, exigindo, em alguns casos, reconstruções periódicas. A KD-Tree é amplamente utilizada em sistemas de informação geográfica (GIS), processamento de imagens, algoritmos de aprendizado de máquina baseados em vizinhança (k -NN), computação gráfica e robótica.

F. Diferenças em relação a BST e AVL

As cinco estruturas estudadas divergem da BST/AVL em pelo menos um de três eixos: (i) a chave manipulada é uma string decomposta em símbolos (Trie, Patricia), e não um valor atômico comparável diretamente; (ii) o balanceamento é obtido por um mecanismo alternativo ao fator de balanceamento explícito da AVL – reorganização amortizada por acesso (Splay) ou aleatoriedade de prioridades (Treap); ou (iii) a chave é multidimensional, exigindo alternância do critério de comparação por nível (KD-Tree). A AVL garante $O(\log n)$ no pior caso à custa de rotações determinísticas a cada desbalanceamento; Splay e Treap trocam essa garantia estrita por simplicidade de implementação, aceitando complexidade amortizada ou esperada, respectivamente.

II. PROJETO E IMPLEMENTAÇÃO DAS ESTRUTURAS

As cinco estruturas foram implementadas em C++17, cada uma em um arquivo próprio (Splay.cpp, Treap.cpp, AvoreTrie.cpp, patricia.cpp, KDTree.cpp), compartilhando um módulo comum (Arvores.h / Padronizar.cpp) responsável pela leitura e normalização dos arquivos de entrada e pela gravação das medições de tempo em CSV. Todas seguem o mesmo padrão de execução: leem um arquivo de dados, executam operações em lote medindo o tempo com `std::chrono`, e exportam uma demonstração pequena em formato Graphviz (.dot) para inspeção visual.

A. Árvore Patricia

Cada nó armazena um mapa de filhos indexado pelo primeiro caractere do rótulo da aresta (`std::map<char, NoPatricia*>`), e cada aresta carrega uma subcadeia (`std::string`) em vez de um único símbolo. A inserção calcula o maior prefixo comum entre a chave e o rótulo de uma aresta existente; quando esse prefixo é parcial, a aresta é dividida em dois nós, um dos quais passa a ter exatamente dois filhos, preservando o invariante de compactação. A remoção localiza o nó terminal, desmarca-o e, quando o nó resultante passa a ter um único filho e não é mais terminal, funde a aresta remanescente com a de seu filho único, restaurando a compactação. O Algoritmo 1: descreve a divisão de nós (*split*) realizada durante a inserção.

Algoritmo 1: Divisão de Nós (Split) e Fusão (Merge) na Patricia Trie

Require: Nó atual N , palavra restante P

```
1:  $idx \leftarrow P[0] - 'a'$ 
2:  $filho \leftarrow N.filho[idx]$ 
3: if  $filho = \text{NULL}$  then
4:    $N.filho[idx] \leftarrow \text{novo Nó}(P, \text{isWord} = \text{true})$ 
5:   return
6: end if
7:  $k \leftarrow \text{calcularPrefixoComum}(P, filho.rotulo)$ 
8: if  $k < |filho.rotulo|$  then
9:    $prefixo \leftarrow filho.rotulo[0 \dots k - 1]$ 
10:   $fimRotulo \leftarrow filho.rotulo[k \dots |filho.rotulo| - 1]$ 
11:   $meio \leftarrow \text{novo Nó}(prefixo, \text{isWord} = \text{false})$ 
12:   $filho.rotulo \leftarrow fimRotulo$ 
13:   $meio.filho[fimRotulo[0] - 'a'] \leftarrow filho$ 
14:  if  $k = |P|$  then
15:     $meio.isWord \leftarrow \text{true}$ 
16:  else
17:     $meio.filho[P[k] - 'a'] \leftarrow \text{novo Nó}(P[k \dots |P| - 1], \text{true})$ 
18:  end if
19:   $N.filho[idx] \leftarrow meio$ 
20: end if
```

B. Árvore Trie

Cada nó mantém um vetor fixo de 26 ponteiros ($\text{NoTrie} \star \text{filhos}[26]$), um por letra do alfabeto (as palavras de entrada são normalizadas – minúsculas, sem acentuação – antes da inserção), e uma marcação booleana de fim de palavra. A inserção percorre a palavra caractere a caractere, criando nós ausentes; a busca por prefixo reaproveita o mesmo percurso, retornando verdadeiro assim que todos os caracteres do prefixo são consumidos, independentemente da marcação de fim de palavra do nó final. A remoção desmarca o nó terminal e libera nós que ficam sem filhos e sem marcação, subindo pela árvore até encontrar um nó ainda necessário. O Algoritmo 2: resume a inserção e a busca exata.

Algoritmo 2: Inserção e Busca na Árvore Trie

Require: Nó raiz R , palavra W

```
1: procedimento Inserir( $R, W$ )
2:    $curr \leftarrow R$ 
3:   para cada  $c \in W$  faça
4:      $idx \leftarrow \text{tolower}(c) - 'a'$ 
5:     se  $curr.children[idx] = \text{NULL}$  então
6:        $curr.children[idx] \leftarrow \text{novo Nó Trie}()$ 
7:     fim se
8:      $curr \leftarrow curr.children[idx]$ 
9:   fim para
10:   $curr.isWord \leftarrow \text{true}$ 
11: fim procedimento
12: função Buscar( $R, W$ )
13:   $curr \leftarrow R$ 
14:  para cada  $c \in W$  faça
15:     $idx \leftarrow \text{tolower}(c) - 'a'$ 
16:    se  $curr.children[idx] = \text{NULL}$  então
17:      retorne falso
18:    fim se
19:     $curr \leftarrow curr.children[idx]$ 
20:  fim para
21:  retorne  $curr.isWord$ 
22: fim função
```

C. Árvore Splay

O nó guarda apenas chave e dois ponteiros de filho. A operação central, $\text{splay}(raiz, chave)$, é recursiva e implementa os três casos clássicos (*zig*, *zig-zig*, *zig-zag*) por meio de rotações simples à esquerda e à direita. Tanto a busca quanto a inserção e a remoção chamam splay antes de qualquer outra ação – inclusive buscas malsucedidas reorganizam a árvore, trazendo o último nó visitado no caminho para a raiz. A remoção separa a árvore em duas subárvores ao redor do nó removido e as funde trazendo o maior elemento da subárvore esquerda para o topo via um novo splay . O Algoritmo 3: detalha a reorganização top-down clássica de Sleator e Tarjan.

Algoritmo 3: Algoritmo de Reorganização Splay (Sleator-Tarjan)

Require: Nó atual N , chave k

```
1: if  $N = \text{NULL} \vee N.\text{chave} = k$  then
2:   return  $N$ 
3: end if
4: if  $k < N.\text{chave}$  then
5:   if  $N.\text{left} = \text{NULL}$  then
6:     return  $N$ 
7:   end if
8:   if  $k < N.\text{left}.\text{chave}$  then
9:     {Zig-Zig Esquerda-Esquerda}
10:     $N.\text{left}.\text{left} \leftarrow \text{Splay}(N.\text{left}.\text{left}, k)$ 
11:     $N \leftarrow \text{RotacaoDireita}(N)$ 
12:   else if  $k > N.\text{left}.\text{chave}$  then
13:     {Zig-Zag Esquerda-Direita}
14:     $N.\text{left}.\text{right} \leftarrow \text{Splay}(N.\text{left}.\text{right}, k)$ 
15:    if  $N.\text{left}.\text{right} \neq \text{NULL}$  then
16:       $N.\text{left} \leftarrow \text{RotacaoEsquerda}(N.\text{left})$ 
17:    end if
18:   end if
19:   if  $N.\text{left} = \text{NULL}$  then
20:     return  $N$ 
21:   else
22:     return  $\text{RotacaoDireita}(N)$ 
23:   end if
24: else
25:   if  $N.\text{right} = \text{NULL}$  then
26:     return  $N$ 
27:   end if
28:   if  $k < N.\text{right}.\text{chave}$  then
29:     {Zig-Zag Direita-Esquerda}
30:     $N.\text{right}.\text{left} \leftarrow \text{Splay}(N.\text{right}.\text{left}, k)$ 
31:    if  $N.\text{right}.\text{left} \neq \text{NULL}$  then
32:       $N.\text{right} \leftarrow \text{RotacaoDireita}(N.\text{right})$ 
33:    end if
34:   else if  $k > N.\text{right}.\text{chave}$  then
35:     {Zig-Zig Direita-Direita}
36:     $N.\text{right}.\text{right} \leftarrow \text{Splay}(N.\text{right}.\text{right}, k)$ 
37:     $N \leftarrow \text{RotacaoEsquerda}(N)$ 
38:   end if
39:   if  $N.\text{right} = \text{NULL}$  then
40:     return  $N$ 
41:   else
42:     return  $\text{RotacaoEsquerda}(N)$ 
43:   end if
44: end if
```

D. Árvore Treap (Tree + Heap)

O nó guarda chave, prioridade (`int`, sorteada no momento da inserção com um gerador `std::mt19937` de seed fixa, para reprodutibilidade) e dois filhos. A inserção desce como em uma BST comum e, na volta da recursão, aplica rotações sempre que um filho tiver prioridade maior que o pai, restaurando a propriedade de heap. A remoção rotaciona

o nó a remover para baixo, sempre na direção do filho de maior prioridade, até que ele se torne uma folha, quando é simplesmente desconectado. O Algoritmo 4: ilustra a inserção; a remoção segue a lógica simétrica, rotacionando o nó-alvo para baixo em vez de para cima.

Algoritmo 4: Inserção e Remoção por Rebaixamento na Treap

Require: Nó atual N , chave k , prioridade p

```
1: if  $N = \text{NULL}$  then
2:   return novo Nó Treap( $k, p$ )
3: end if
4: if  $k = N.\text{chave}$  then
5:   return  $N$  {Rejeita duplicatas}
6: end if
7: if  $k < N.\text{chave}$  then
8:    $N.\text{esquerda} \leftarrow \text{Inserir}(N.\text{esquerda}, k, p)$ 
9:   if  $N.\text{esquerda}.\text{prioridade} > N.\text{prioridade}$  then
10:     $N \leftarrow \text{RotacaoDireita}(N)$ 
11:   end if
12: else
13:    $N.\text{direita} \leftarrow \text{Inserir}(N.\text{direita}, k, p)$ 
14:   if  $N.\text{direita}.\text{prioridade} > N.\text{prioridade}$  then
15:     $N \leftarrow \text{RotacaoEsquerda}(N)$ 
16:   end if
17: end if
18: return  $N$ 
```

E. KD-Tree (k -dimensional Tree)

O ponto é representado por `std::vector<double>`, e a árvore recebe a dimensão k no construtor – inferida automaticamente, no `main()`, pela quantidade de números na primeira linha do arquivo de entrada (generalização que permite usar a mesma implementação para 2D, 3D ou mais, sem alterar código). Cada nível de profundidade compara a coordenada de índice `profundidade % k`. A busca de vizinho mais próximo utiliza poda por hiperplano: desce primeiro pelo lado mais provável e só visita o lado oposto se a distância ao hiperplano de corte for menor que a melhor distância já encontrada – é essa poda que evita uma varredura completa da árvore. O Algoritmo 5: ilustra o princípio análogo de poda espacial aplicado a uma busca por alcance (*range search*) bidimensional.

Algoritmo 5: Busca por Alcance 2D na KD-Tree

Require: Nó N , limites $minP, maxP$, profundidade d

```

1: if  $N = \text{NULL}$  then
2:   return
3: end if
4: if  $N.ponto \in [minP, maxP]$  then
5:   adicionar  $N.ponto$  aos resultados
6: end if
7:  $eixo \leftarrow d \pmod{2}$ 
8: if  $minP[eixo] \leq N.ponto[eixo]$  then
9:   RangeSearch( $N.left, minP, maxP, d + 1$ )
10: end if
11: if  $maxP[eixo] \geq N.ponto[eixo]$  then
12:   RangeSearch( $N.right, minP, maxP, d + 1$ )
13: end if

```

III. DEMONSTRAÇÃO E RASTREAMENTO VISUAL

Para cada estrutura, uma instância pequena e fixa (10 elementos) foi construída especificamente para gerar diagramas legíveis, exportados em formato Graphviz (.dot) diretamente pelo código C++ e renderizados com o utilitário dot. Optou-se por esse conjunto reduzido porque a árvore completa (centenas a milhares de elementos, usada nas medições de tempo da Seção V) resulta em diagramas ilegíveis.

A Figura 5 ilustra bem o contraste central entre Trie e Patricia: a Trie precisa de um nó por caractere percorrido, enquanto a Patricia comprime caminhos sem ramificação em uma única aresta rotulada – para o mesmo conjunto de 10 palavras, a Patricia (Figura 4) tem profundidade máxima 3, contra profundidade máxima 9 da Trie (Figura 3).

IV. ANÁLISE DE COMPLEXIDADE E COMPARAÇÃO TEÓRICA

A Tabela I resume a complexidade assintótica das operações fundamentais de cada estrutura, incluindo BST e AVL como referência.

Tabela I
COMPLEXIDADE ASSINTÓTICA DAS OPERAÇÕES FUNDAMENTAIS

Estrutura	Busca	Inserção	Remoção	Construção	Espaço
BST (não balanceada)	$O(n)$ pior	$O(n)$ pior	$O(n)$ pior	$O(n \log n)$	$O(n)$
AVL	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(n \log n)$	$O(n)$
Trie	$O(m)$	$O(m)$	$O(m)$	$O(n \cdot m)$	$O(n \cdot m \cdot \Sigma)$
Patricia	$O(m)$	$O(m)$	$O(m)$	$O(n \cdot m)$	$O(n \cdot m)$
Splay	$O(\log n)^*$	$O(\log n)^*$	$O(\log n)^*$	$O(n \log n)$	$O(n)$
Treap	$O(\log n)^\dagger$	$O(\log n)^\dagger$	$O(\log n)^\dagger$	$O(n \log n)$	$O(n)$
KD-Tree (k -D)	$O(\log n)^\ddagger$	$O(\log n)^\ddagger$	$O(n)$	$O(n \log n)$	$O(n)$

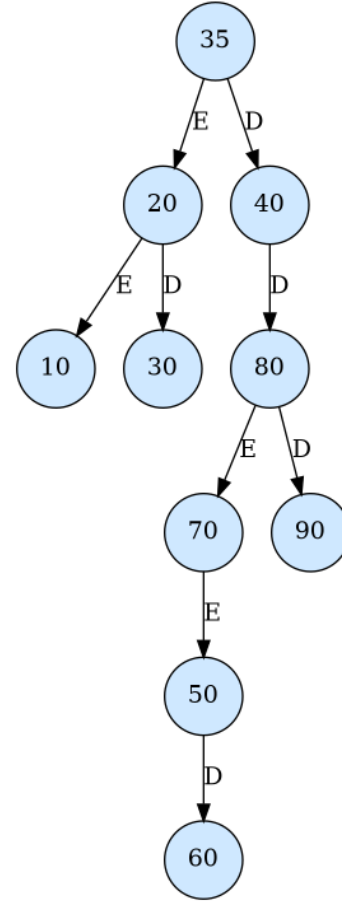
$n = n^\circ$ de chaves, $m =$ comprimento da chave, $|\Sigma| =$ tamanho do alfabeto.

*amortizado; pior caso de uma única operação é $O(n)$. † esperado (aleatoriedade das prioridades).

‡ médio; degrada para $O(n)$ com dimensionalidade alta ou dados adversariais.

Por que Trie e Patricia dependem de m , não de n : em ambas, o custo de uma operação é proporcional ao número de símbolos percorridos até localizar ou inserir a chave, e não à quantidade de chaves já armazenadas – por isso a busca não degrada com o crescimento de n , ao contrário de uma BST. A diferença entre as duas está no espaço: a Trie aloca um nó por símbolo mesmo em caminhos sem ramificação, enquanto a Patricia colapsa esses caminhos, tornando o número de nós

Splay - ANTES de remover a chave 40



Splay - DEPOIS de remover a chave 40

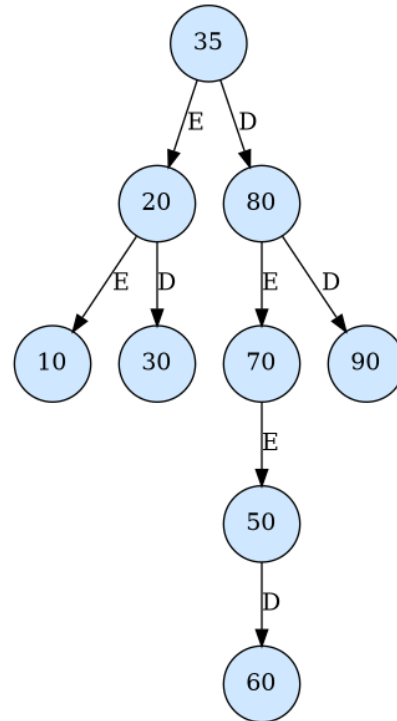
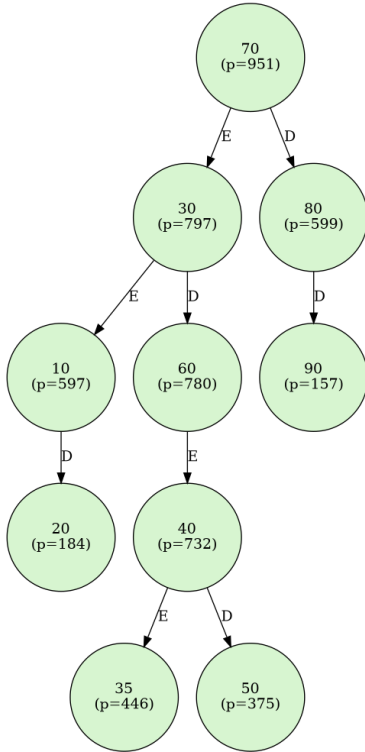


Figura 1. Árvore Splay antes e depois da remoção da chave 40. A remoção provoca um *splay* do sucessor (chave 35) até a posição do nó removido, reorganizando toda a subárvore à direita de 20.

Treap - ANTES de remover a chave 40



Treap - DEPOIS de remover a chave 40

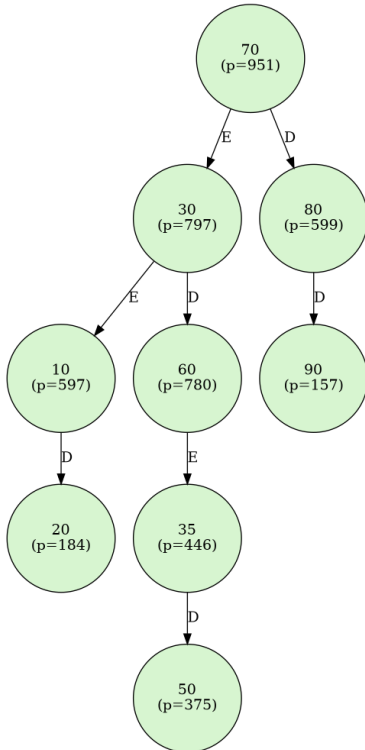


Figura 2. Árvore Treap antes e depois da remoção da chave 40, com a prioridade (p) de cada nó explícita. A remoção rotaciona o nó 40 para baixo em direção ao filho de maior prioridade até virar folha.

Trie - ANTES de remover "carro"
(circulo duplo laranja = fim de palavra)

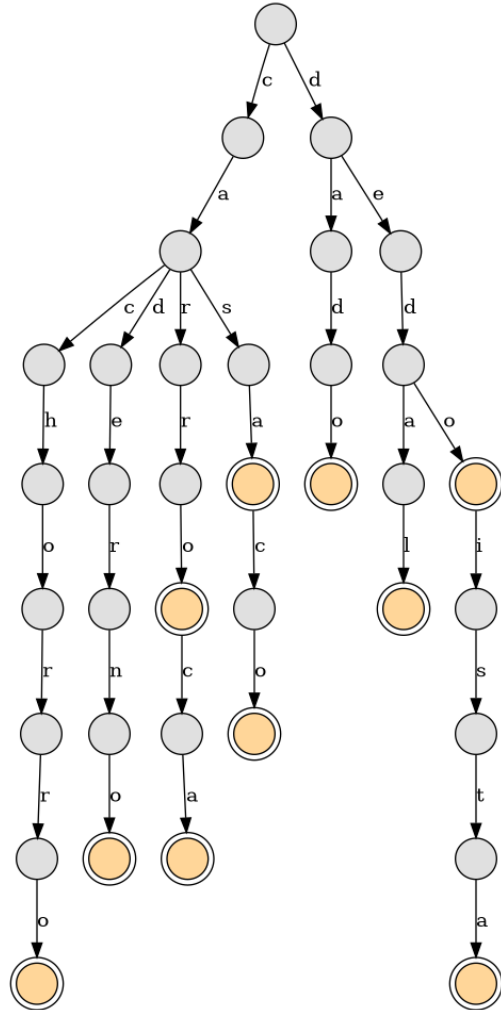


Figura 3. Trie construída a partir de 10 palavras com prefixos compartilhados. Cada aresta representa um único caractere; círculos duplos em laranja marcam nós de fim de palavra. Note o compartilhamento dos prefixos *ca* e *delda*.

Patricia - ANTES de remover "carro"
(circulo duplo laranja = fim de palavra; rotulo da aresta = trecho compactado)

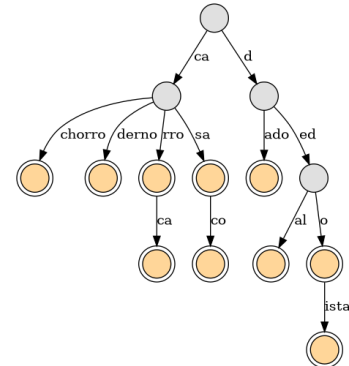


Figura 4. Árvore Patricia para o mesmo conjunto de 10 palavras da Figura 3. Arestas carregam subcadeias inteiras (ex.: *chorro*, *derno*, *ista*), resultando em uma árvore consideravelmente mais rasa que a Trie equivalente.

V. METODOLOGIA EXPERIMENTAL E RESULTADOS

A. Metodologia

Cada estrutura foi medida com `std::chrono::high_resolution_clock`, cronometrando blocos completos de operações (não operações isoladas, para reduzir ruído de medição). Splay, Treap e KD-Tree foram testados em cinco tamanhos de entrada ($N = 100, 1000, 10000, 100000$ e, para a KD-Tree, também 200000 pontos), gerados por um script determinístico (seed fixa) que reproduz os três padrões de dados originais: para a Splay, um bloco de inserção em ordem aleatória seguido de $5N$ acessos com 70% concentrados em um subconjunto "quente" de 5% das chaves (para evidenciar o efeito do *splaying*); para a Treap, os cenários RANDOM, SORTED e REVERSE; para a KD-Tree, pontos uniformemente distribuídos e pontos concentrados em agrupamentos, em $[0, 100]^2$. Trie e Patricia foram medidas com o conjunto de 1000 palavras fornecido, em inserção, busca e remoção de metade das palavras. Todas as medições foram gravadas em CSV (estrutura, bloco, operação, quantidade, tempo em ms), permitindo reconstrução e comparação posterior.

B. Resultados

A Figura 7 mostra o tempo de inserção de Splay, Treap e KD-Tree em função de N , em escala log-log.

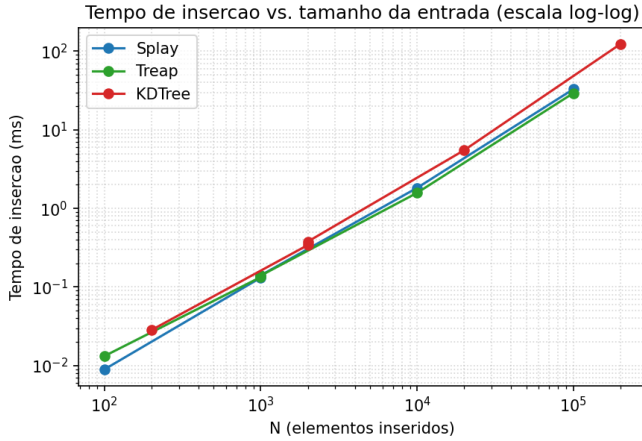


Figura 7. Tempo de inserção vs. tamanho da entrada (N), escala log-log, para Splay (ordem aleatória), Treap (cenário RANDOM) e KD-Tree (pontos uniformes em 2D).

A Tabela II resume uma amostra dos tempos medidos para $N = 1000$ (valor comum entre as cinco estruturas), permitindo comparação direta.

C. Discussão

O gráfico log-log da Figura 7 mostra as três curvas próximas de uma reta com inclinação levemente superior a 1, coerente com o comportamento teórico $O(n \log n)$ esperado para a construção de todas as três estruturas – o crescimento super-linear observado (o tempo aumenta pouco mais que proporcionalmente a N) é justamente a assinatura do fator

Tabela II
TEMPO DE EXECUÇÃO PARA $N = 1000$ (OU 1000 PALAVRAS, PARA TRIE/PATRICIA)

Estrutura	Inserção (ms)	Busca (ms)	Remoção (ms)
Splay	0,141	0,298 ^a	0,004 ^b
Treap (RANDOM)	0,134	0,036	0,067 ^c
Trie	0,249	0,012	0,056 ^c
Patricia	0,334	0,172	0,130 ^c

a: 5000 buscas (bloco ACCESS). b: 50 remoções (bloco HOTSET). c: remoção de metade das chaves/palavras (500).

log n . A KD-Tree consistentemente apresenta o maior tempo de inserção entre as três, mesmo em 2D; isso é coerente com o custo por nó ser maior (cada inserção compara vetores de coordenadas, não um único inteiro).

A busca na Trie (0,012 ms para 1000 buscas) é a mais rápida da tabela, consistente com sua complexidade $O(m)$ independente de n : para as palavras de tamanho moderado usadas no experimento, o custo por busca é pequeno e não cresce com o tamanho do dicionário. A Patricia, apesar de ter a mesma complexidade assintótica, apresentou busca mais lenta que a Trie no experimento; a causa provável é o custo constante mais alto por nó (comparação de subcadeias e uso de `std::map` para indexar filhos, em vez de acesso direto por índice de vetor como na Trie) – um lembrete de que complexidade assintótica igual não implica desempenho prático igual.

A remoção na Splay (0,004 ms para 50 chaves do HOTSET) é discrepante em relação às demais remoções da tabela porque essas 50 chaves já haviam sido splayadas para perto da raiz pelo bloco ACCESS imediatamente anterior (por construção, o HOTSET é justamente o subconjunto mais acessado) – um exemplo direto, e mensurável, do princípio de localidade temporal que fundamenta a árvore Splay.

VI. APLICAÇÕES, ANÁLISE CRÍTICA E DISCUSSÃO

A. Aplicações

A **Trie** é a estrutura de escolha para autocompletar e correção ortográfica, onde a busca por prefixo em $O(m)$ permite listar sugestões instantaneamente conforme o usuário digita. A **Patricia** é preferível quando o volume de chaves é grande e a memória é escassa – seu uso canônico é a busca por prefixo mais longo em tabelas de roteamento IP, onde milhões de prefixos precisam caber em memória de roteador. A **Splay** se destaca em sistemas de cache e estruturas com padrão de acesso desigual (poucos itens concentram a maioria dos acessos), como caches de tradução de páginas de memória. A **Treap** é atrativa quando se precisa de um conjunto ordenado com operações eficientes de *split/merge* e implementação simples, como em estruturas de dados persistentes e balanceamento de carga distribuído. A **KD-Tree** é a base de buscas por vizinho mais próximo em sistemas de informação geográfica, algoritmos de aprendizado de máquina baseados em vizinhança (k -NN) e detecção de colisão em computação gráfica.

B. Vantagens e limitações observadas

A principal vantagem prática da Treap, confirmada na implementação, é a simplicidade: o balanceamento inteiro decorre de uma comparação de prioridade, sem precisar manter ou recalculer altura/fator de balanceamento como na AVL. Em contrapartida, a Splay exigiu mais cuidado de implementação: os três casos de rotação (*zig*, *zig-zig*, *zig-zag*) para cada lado (esquerda/direita) totalizam uma lógica consideravelmente mais longa e propensa a erros de ponteiro do que a Treap. Entre Trie e Patricia, a Patricia exigiu a lógica mais delicada de todo o trabalho: a divisão de uma aresta existente ao inserir uma chave que compartilha apenas um prefixo parcial com o rótulo daquela aresta precisa tratar corretamente os casos em que o prefixo comum é vazio, total ou parcial.

A generalização da KD-Tree para dimensão k arbitrária (originalmente fixa em 2D) foi direta porque a lógica de alternância de eixo (profundidade $\% k$) já era escrita de forma genérica; a única mudança estrutural foi trocar coordenadas fixas (`double x, y`) por um vetor de tamanho variável.

C. Teoria vs. experimento

Os resultados experimentais da Seção V foram, de forma geral, coerentes com a análise teórica da Seção IV: o crescimento observado do tempo de inserção acompanhou o formato esperado de $O(n \log n)$ para as três estruturas testadas em múltiplos tamanhos. A principal divergência notada foi entre Trie e Patricia na operação de busca: a teoria prevê complexidade idêntica ($O(m)$ para ambas), mas o experimento mostrou a Patricia consistentemente mais lenta – evidenciando que constantes ocultas pela notação assintótica (custo de comparação de subcadeias, indireção do `std::map`) podem dominar o tempo real para o tamanho de entrada testado.

D. Possíveis extensões

Entre as melhorias mais relevantes: (i) implementar reconstrução periódica ou rebalanceamento incremental na KD-Tree, hoje ausente, para evitar degradação após muitas remoções; (ii) substituir o `std::map<char, NoPatricia*>` por um vetor indexado, reduzindo a constante por nó da Patricia; (iii) estender os experimentos a distribuições adversariais (por exemplo, sequência de acessos adversarial contra a Splay) para observar o pior caso $O(n)$ isolado, hoje mascarado pela média amortizada.

VII. CONCLUSÃO

As cinco estruturas estudadas resolvem, cada uma, uma limitação distinta das BSTs balanceadas clássicas. Trie e Patricia trocam a comparação de chaves atômicas por decomposição em símbolos, tornando o custo por operação proporcional ao comprimento da chave em vez do tamanho do conjunto – a Patricia é estritamente mais econômica em espaço, à custa de uma implementação mais delicada. Splay e Treap alcançam balanceamento sem o custo de manutenção explícita da AVL,

trocando garantia de pior caso por garantia amortizada (Splay) ou esperada (Treap); entre as duas, a Treap se mostrou a mais simples de implementar corretamente. A KD-Tree resolve um problema ortogonal às demais – indexação multidimensional –, mas é a única cuja complexidade de busca degrada de forma mensurável com o crescimento da dimensionalidade.

Os experimentos confirmaram o comportamento assintótico esperado para inserção nas três estruturas testadas em escala (Splay, Treap, KD-Tree), com o gráfico log-log evidenciando o fator $\log n$ do custo teórico. A principal lição prática do trabalho é que complexidade assintótica idêntica (caso de Trie e Patricia na busca) não garante desempenho idêntico: a constante por operação, determinada por decisões de implementação como a estrutura de indexação de filhos, pode dominar o tempo observado para tamanhos de entrada realistas.

Quanto à escolha de estrutura para um problema específico: a Trie é preferível quando o alfabeto é pequeno e a velocidade de busca por prefixo é crítica; a Patricia, quando a memória é a restrição dominante; a Splay, quando o padrão de acesso é desigual e previsivelmente concentrado; a Treap, quando se deseja um conjunto ordenado balanceado com implementação simples; e a KD-Tree, sempre que a chave for inerentemente multidimensional e a dimensionalidade for baixa a moderada.

REFERÊNCIAS

- [1] BENTLEY, J. L. Multidimensional Binary Search Trees Used for Associative Searching. *Communications of the ACM*, v. 18, n. 9, p. 509–517, 1975.
- [2] CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. *Introduction to Algorithms*. 3. ed. Cambridge: MIT Press, 2009.
- [3] FREDKIN, E. Trie Memory. *Communications of the ACM*, v. 3, n. 9, p. 490–499, 1960.
- [4] MORRISON, D. R. PATRICIA — Practical Algorithm To Retrieve Information Coded in Alphanumeric. *Journal of the ACM*, v. 15, n. 4, p. 514–534, 1968.
- [5] SEDGEWICK, R.; WAYNE, K. *Algorithms*. 4. ed. Boston: Addison-Wesley, 2011.
- [6] SEIDEL, R.; ARAGON, C. R. Randomized Search Trees. *Algorithmica*, v. 16, n. 4/5, p. 464–497, 1996.
- [7] SLEATOR, D. D.; TARJAN, R. E. Self-Adjusting Binary Search Trees. *Journal of the ACM*, v. 32, n. 3, p. 652–686, 1985.
- [8] VUILLEMIN, J. A Unifying Look at Data Structures. *Communications of the ACM*, v. 23, n. 4, p. 229–239, 1980.
- [9] FARIA, G. A. *Trabalho 1 – Estruturas em Árvores Avançadas*. GitHub, 2026. Disponível em: <https://github.com/SEU-USUARIO/SEU-REPOSITORIO>. Acesso em: 19 set. 2026.